

Modeling Player Performance by Time

October 23, 2025

1 Modeling Tennis Player Performance Over Their Career

In this notebook, we explore how a tennis player's performance evolves over the course of their career. Tennis players typically have an average career span of around **15 years**, during which their performance rises, peaks, and eventually declines.

We model this trajectory using a **polynomial function**. To fit the polynomial to our synthetic data, we will use **gradient descent**, an iterative optimization method that adjusts the polynomial coefficients to minimize prediction error.

The function and gradient computations are implemented using **JAX**, a Python library developed by Google. JAX is particularly useful here because it provides **automatic differentiation**, allowing us to compute gradients of the loss function efficiently and accurately. It also supports **vectorized operations** and ensures reproducibility, which makes gradient descent both fast and reliable.

This synthetic data lets us simulate realistic career patterns and analyze trends in player development, peak performance, and late-career decline.

1.0.1 Step 1: Generating Synthetic Data (5 Player Careers)

```
[5]: import jax.numpy as jnp
import jax.random as random
```

```
[32]: key = random.PRNGKey(87)  # initial key (reproducibility)

# 15-year career, measured twice per year (30 data points are measured per
#   ↪ player)
time = jnp.linspace(0, 15, 30)

# Typical tennis career: polynomial coefficients chosen to rise, peak, then
#   ↪ decline
coefficients = jnp.array([-0.04, 0.6, 0.4, 60.0]) # a, b, c, d
# Explanation (idea behind the data):
# a negative cubic gives a slight decline after peak,
# b positive quadratic makes the rise steep,
# c linear small growth,
# d starting performance around 50.
```

```

def generate_career_curve(x, coeffs, noise_scale, key):
    a, b, c, d = coeffs
    y = a*x**3 + b*x**2 + c*x + d

    # adding realistic noise
    noise = noise_scale * random.normal(subkey, shape=x.shape)
    # more noise scale = more noisy data
    # random numbers are generated to be multiplied to player's yearly
    ↪performance
    # these random numbers have mean 0 and std 1 (normal distribution)
    # generating a different key every time (key = reproducibility ID) so no
    ↪player is the same in terms of career path
    y = y + noise

    return y

num_players = 5 # five players are our training and testing data
y_players = []
for i in range(num_players):
    key, subkey = random.split(key) # split to get unique key per player
    y = generate_career_curve(time, coefficients, 2.0, subkey)
    y_players.append(y)

# we have gathered 5 player's worth of synthetic data (5 careers)
y_players = jnp.stack(y_players)

```

[33]: y_players

```

[33]: Array([[60.782272, 60.83285 , 62.270523, 61.251465, 61.05171 , 65.51866 ,
        67.6711 , 65.87225 , 68.42586 , 70.36303 , 72.03496 , 72.812706,
        73.54668 , 75.9327 , 80.71456 , 80.2062 , 80.6142 , 81.160034,
        83.29717 , 80.35644 , 85.16837 , 82.09335 , 84.962425, 84.414696,
        76.7649 , 81.135826, 75.736984, 74.45473 , 72.819954, 65.58934 ],
        [62.47101 , 62.79577 , 60.506844, 62.793667, 62.90146 , 65.10926 ,
        63.727848, 67.96403 , 67.25607 , 67.74189 , 73.99916 , 71.26984 ,
        76.56182 , 80.12798 , 76.854805, 80.051155, 83.273315, 82.28728 ,
        84.523705, 81.798744, 81.82443 , 83.939415, 81.916565, 78.90796 ,
        80.25252 , 76.099884, 79.95949 , 71.97867 , 70.40746 , 65.58913 ],
        [60.65575 , 58.017776, 59.33299 , 61.09905 , 63.316204, 62.93715 ,
        67.17679 , 69.712234, 69.97514 , 72.741486, 70.82139 , 72.77779 ,
        75.04392 , 79.58265 , 80.96538 , 80.68634 , 83.815636, 82.80609 ,
        83.49113 , 86.302864, 81.30843 , 84.35815 , 83.593796, 82.9504 ,
        80.78667 , 78.83486 , 75.62057 , 70.660835, 67.5898 , 64.45134 ],
        [58.909344, 59.039555, 64.133804, 63.663548, 60.884426, 62.49519 ,
        66.58897 , 69.377205, 70.974525, 70.70134 , 72.17614 , 75.750885,
        71.66358 , 76.061676, 81.60247 , 81.299706, 82.45581 , 80.49081 ,
        81.98516 , 82.848434, 85.0608 , 83.15833 , 81.272934, 82.266075,

```

```

80.15192 , 81.11511 , 80.58417 , 73.074905, 67.38384 , 68.29016 ],
[56.298355, 62.473816, 59.20397 , 64.62705 , 58.449482, 62.811703,
63.911697, 68.65894 , 64.77342 , 73.15888 , 70.477554, 74.61614 ,
75.73194 , 79.78747 , 78.31697 , 79.96523 , 82.35009 , 82.62978 ,
83.228874, 83.29202 , 81.499626, 85.4126 , 83.09438 , 83.348564,
81.52196 , 80.686104, 76.071785, 75.910324, 71.144516, 63.558002]],
dtype=float32)

```

1.0.2 Step 2: Fitting Model to Synthetic Player Data

Model uses gradient descent in JAX to learn player career performance paths.

```

[46]: from jax import grad

t_scaled = time / 15.0 # scale career years to [0,1] for stability so t goes
    ↪ from 0 to 15

# Polynomial model
def model(params, time):
    a, b, c, d = params
    return a*time**3 + b*time**2 + c*time + d

# Mean squared error loss - avg squared difference between predicted and real
    ↪ values
def loss_fn(params, time, performances):
    preds = model(params, time)
    error = preds - performances # error for each player
    return jnp.mean(error**2) # avg loss across all players

# JAX computes the gradient of the loss with respect to parameters
grad_loss_fn = grad(loss_fn)

def fit_polynomial_gradient(time, performances, lr=0.001, steps=5000):
    # Initialize parameters at 0
    params = jnp.array([0.0, 0.0, 0.0, 0.0])
    # the model is trying to learn the real coefficients (model's guessed
    ↪ coefficients = params)

    for i in range(steps):
        grads = grad_loss_fn(params, time, performances)
        # returns array of 4 numbers each showing how loss changes if we
        ↪ increase each param

        params = params - lr * grads
        # update params in the opposite direction of the gradient to reduce the
        ↪ loss next time

```

```

        if i % 500 == 0: # print every 500 steps (iteration number and current
↪loss)
            print(f"Step {i}, Loss: {loss_fn(params, time, performances):.4f}")

        return params

params = fit_polynomial_gradient(t_scaled, y_players, lr=1e-2, steps=10000)

print("Fitted coefficients for all players:", params)

```

```

Step 0, Loss: 5177.6167
Step 500, Loss: 40.5725
Step 1000, Loss: 30.9804
Step 1500, Loss: 27.4720
Step 2000, Loss: 24.5840
Step 2500, Loss: 22.1050
Step 3000, Loss: 19.9741
Step 3500, Loss: 18.1420
Step 4000, Loss: 16.5669
Step 4500, Loss: 15.2123
Step 5000, Loss: 14.0473
Step 5500, Loss: 13.0453
Step 6000, Loss: 12.1831
Step 6500, Loss: 11.4412
Step 7000, Loss: 10.8025
Step 7500, Loss: 10.2527
Step 8000, Loss: 9.7790
Step 8500, Loss: 9.3709
Step 9000, Loss: 9.0191
Step 9500, Loss: 8.7156
Fitted coefficients for all players: [-36.875275      0.12246084  50.586166
57.864986 ]

```

1.0.3 Step 3: Comparing the Model's Understanding to the Real Deal

```

[47]: import matplotlib.pyplot as plt

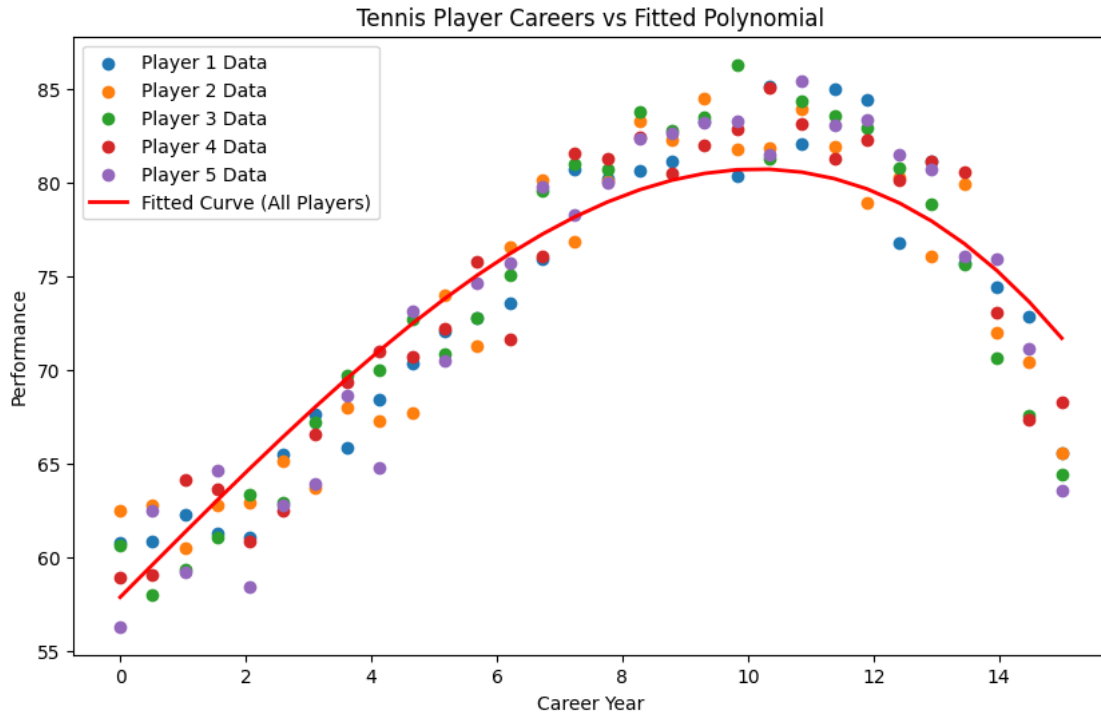
plt.figure(figsize=(10,6))

# Plot each player's data
for i in range(y_players.shape[0]):
    plt.scatter(time, y_players[i], label=f"Player {i+1} Data")

# Plot the single fitted curve
y_fit = model(params, t_scaled)
plt.plot(time, y_fit, color='red', linewidth=2, label="Fitted Curve (All
↪Players)")

```

```
plt.xlabel("Career Year")
plt.ylabel("Performance")
plt.title("Tennis Player Careers vs Fitted Polynomial")
plt.legend()
plt.show()
```



1.0.4 Step 4: Model Predict's a New Player's Path (Untrained)

```
[49]: # Generate a new player using the same function
key, subkey = random.split(key) # get a new unique subkey
y_new_player = generate_career_curve(time, coefficients, noise_scale=2.0,
    ↪key=subkey)
```

```
[53]: import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))

# Plot existing players
for i in range(y_players.shape[0]):
    plt.scatter(time, y_players[i], label=f"Player {i+1} Data")

# Plot fitted average curve
```

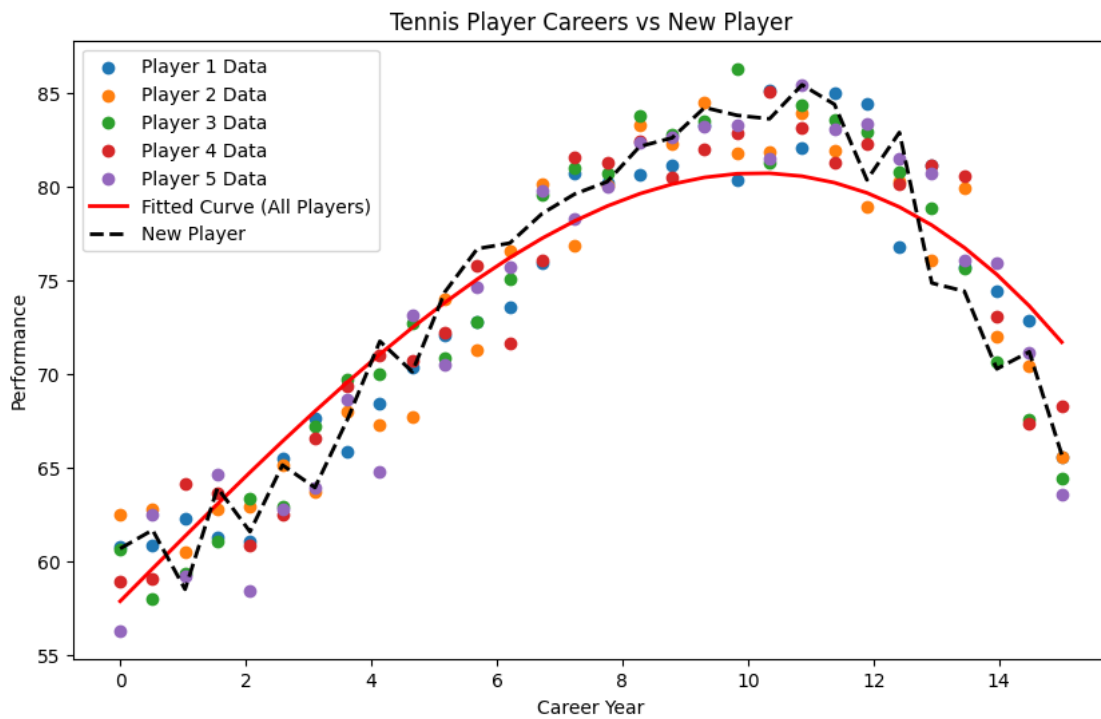
```

y_fit = model(params, t_scaled)
plt.plot(time, y_fit, color='red', linewidth=2, label="Fitted Curve (All
↳Players)")

# Plot new player
plt.plot(time, y_new_player, color='black', linewidth=2, linestyle='--',
↳label="New Player")

plt.xlabel("Career Year")
plt.ylabel("Performance")
plt.title("Tennis Player Careers vs New Player")
plt.legend()
plt.show()

```



1.0.5 Conclusion

Overall, the model was simple and easy to run. After initial trouble with gradient explosion (gradient nan values solved by t-scaled and adjusting learning rate), the model began to work and parameters were adjusted to near optimum levels.

The model understood the traditional player career performance trajectory well and successfully modeled that of a new player that it did not see during training.